

**Московский государственный технический
университет им. Н.Э. Баумана**

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”

Курс «Парадигмы и конструкции языков программирования»

Отчет по лабораторной работе №5
«Модульное тестирование в Python»

Выполнил:
студент группы ИУ5 - 32Б:
Акопов А.
Подпись и дата:

Проверил:
преподаватель каф. ИУ5
Нардид А. Н.
Подпись и дата:

Москва, 2025 г.

Задание:

1. Выберите любой фрагмент кода из лабораторных работ 1 или 2 или 3-4.
2. Модифицируйте код таким образом, чтобы он был пригоден для модульного тестирования.
3. Разработайте модульные тесты. В модульных тестах необходимо применить следующие технологии:
 - TDD - фреймворк (не менее 3 тестов).
 - BDD - фреймворк (не менее 3 тестов).
 - Создание Моск-объектов (необязательное дополнительное задание).

Папка tests:

lab5_tests_TDD.py

```
import unittest
import sys
import os
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from lab_python_fp.field import field
from lab_python_fp.unique import Unique
from lab_python_fp.gen_random import gen_random

class TestLabFunctionsTDD(unittest.TestCase):

    def setUp(self):
        self.test_goods = [
            {'title': 'Ковер', 'price': 2000, 'color': 'green'},
            {'title': 'Диван для отдыха', 'price': 5300, 'color': 'black'},
            {'title': None, 'price': 1000},
            {'invalid': 'item'}
        ]

    def test_field_single_key(self):
        """Тест field с одним ключом"""
        result = list(field(self.test_goods, 'title'))
        expected = ['Ковер', 'Диван для отдыха']
        self.assertEqual(result, expected)

    def test_field_multiple_keys(self):
        """Тест field с несколькими ключами"""
        result = list(field(self.test_goods, 'title', 'price'))
        expected = [
            {'title': 'Ковер', 'price': 2000},
            {'title': 'Диван для отдыха', 'price': 5300},
            {'price': 1000}
```

```

    ]
    self.assertEqual(result, expected)

def test_field_empty_list(self):
    """Тест field с пустым списком"""
    result = list(field([], 'title'))
    self.assertEqual(result, [])

def test_unique_numbers(self):
    """Тест Unique с числами"""
    data = [1, 1, 2, 2, 3, 3, 1, 2, 3]
    result = list(Unique(data))
    expected = [1, 2, 3]
    self.assertEqual(result, expected)

def test_unique_strings_case_sensitive(self):
    """Тест Unique с регистрозависимыми строками"""
    data = ['a', 'A', 'b', 'B', 'a', 'A']
    result = list(Unique(data))
    expected = ['a', 'A', 'b', 'B']
    self.assertEqual(result, expected)

def test_unique_strings_case_insensitive(self):
    """Тест Unique с регистронезависимыми строками"""
    data = ['a', 'A', 'b', 'B', 'a', 'A']
    result = list(Unique(data, ignore_case=True))
    expected = ['a', 'b']
    self.assertEqual(result, expected)

def test_gen_random_count(self):
    """Тест gen_random на количество элементов"""
    count = 5
    result = list(gen_random(count, 1, 10))
    self.assertEqual(len(result), count)

def test_gen_random_range(self):
    """Тест gen_random на диапазон значений"""
    begin, end = 1, 5
    result = list(gen_random(10, begin, end))
    for num in result:
        self.assertTrue(begin <= num <= end)

if __name__ == '__main__':
    unittest.main()

```

Панка tests/features:

lab5.feature

Feature: Laboratory Functions Testing

As a developer

I want to test laboratory functions

So that I can ensure they work correctly

Scenario: Extract single field from dictionaries

Given a list of dictionaries with goods

When I extract field "title"

Then I should get list with titles

"""

Ковер, Диван для отдыха

"""

Scenario: Extract multiple fields from dictionaries

Given a list of dictionaries with goods

When I extract fields "title" and "price"

Then the result should contain 3 items

Scenario: Remove duplicates from numbers

Given a list with duplicate items

When I get unique items

Then I should get list with values

```
"""
```

```
1, 2, 3
```

```
"""
```

Панка tests/features/steps:

lab5_tests_BDD.py

```
from behave import given, when, then
import sys
import os
sys.path.append(os.path.join(os.path.dirname(__file__), '../..'))

from lab_python_fp.field import field
from lab_python_fp.unique import Unique

@given('a list of dictionaries with goods')
def step_given_list_of_dicts(context):
    context.goods = [
        {'title': 'Ковер', 'price': 2000, 'color': 'green'},
        {'title': 'Диван для отдыха', 'price': 5300, 'color': 'black'},
        {'title': None, 'price': 1000}
    ]

@given('a list with duplicate items')
def step_given_list_with_duplicates(context):
    context.data = [1, 1, 2, 2, 3, 3, 1, 2, 3]

@given('a list with case sensitive strings')
def step_given_case_sensitive_strings(context):
    context.data = ['a', 'A', 'b', 'B', 'a', 'A']

@when('I extract field "{field_name}"')
def step_when_extract_single_field(context, field_name):
    context.result = list(field(context.goods, field_name))

@when('I extract fields "{field1}" and "{field2}"')
def step_when_extract_multiple_fields(context, field1, field2):
    context.result = list(field(context.goods, field1, field2))

@when('I get unique items')
def step_when_get_unique_items(context):
    context.result = list(Unique(context.data))
```

```

@when('I get unique items ignoring case')
def step_when_get_unique_ignore_case(context):
    context.result = list(Unique(context.data, ignore_case=True))

@then('I should get list with values')
def step_then_should_get_list(context):
    expected = context.text.strip().split(', ')
    expected = [int(x) if x.isdigit() else x for x in expected]
    assert context.result == expected, f"Expected {expected}, got {context.result}"

@then('I should get list with titles')
def step_then_should_get_titles(context):
    expected = context.text.strip().split(', ')
    assert context.result == expected, f"Expected {expected}, got {context.result}"

@then('the result should contain {count:d} items')
def step_then_result_should_contain_count(context, count):
    assert len(context.result) == count, f"Expected {count} items, got {len(context.result)}"

@then('the result should not contain duplicates')
def step_then_no_duplicates(context):
    seen = set()
    for item in context.result:
        key = item.lower() if isinstance(item, str) else item
        assert key not in seen, f"Duplicate found: {item}"
        seen.add(key)

```

Скриншоты работы:

```
(base) an.akopov@MacBook-Air-Akopov lab3 % python3 -m unittest tests.lab5_tests_TDD
.....
-----
Ran 8 tests in 0.000s

OK
(base) an.akopov@MacBook-Air-Akopov lab3 %
```

Уроним тесты

```
(base) an.akopov@MacBook-Air-Akopov lab3 % python3 -m unittest tests.lab5_tests_TDD
.F...F..
=====
FAIL: test_field_multiple_keys (tests.lab5_tests_TDD.TestLabFunctionsTDD.test_field_multiple_keys)
Тест field с несколькими ключами
-----
Traceback (most recent call last):
  File "/Users/an.akopov/Documents/PCPL_Labs/lab3/tests/lab5_tests_TDD.py", line 34, in test_field_multiple_keys
    self.assertEqual(result, expected)
    ~~~~~^~~~~~
AssertionError: Lists differ: [{'ti[20 chars]e': 245}, {'title': 'Диван для отдыха', 'price[21 chars]000}] != [{'ti[20 chars]e': 2000}, {'title': 'Диван для отдыха'
, 'pric[22 chars]000}]

First differing element 0:
{'title': 'Ковер', 'price': 245}
{'title': 'Ковер', 'price': 2000}
- [{'price': 245, 'title': 'Ковер'},
?      ^^
+ [{'price': 2000, 'title': 'Ковер'},
?      ^^^
    {'price': 5300, 'title': 'Диван для отдыха'},
    {'price': 1000}]

=====
FAIL: test_unique_numbers (tests.lab5_tests_TDD.TestLabFunctionsTDD.test_unique_numbers)
Тест Unique с числами
-----
Traceback (most recent call last):
  File "/Users/an.akopov/Documents/PCPL_Labs/lab3/tests/lab5_tests_TDD.py", line 46, in test_unique_numbers
    self.assertEqual(result, expected)
    ~~~~~^~~~~~
AssertionError: Lists differ: [1, 2, 3, 9] != [1, 2, 3]

First list contains 1 additional elements.
First extra element 3:
9
- [1, 2, 3, 9]
?      _
+ [1, 2, 3]

-----
Ran 8 tests in 0.001s
```

```

(base) an.akopov@MacBook-Air-Akopov lab3 % pip3 list | grep behave
(base) an.akopov@MacBook-Air-Akopov lab3 % behave tests/features/
USING RUNNER: behave.runner:Runner
Feature: Laboratory Functions Testing # tests/features/lab5.feature:1
  As a developer
  I want to test laboratory functions
  So that I can ensure they work correctly
  Scenario: Extract single field from dictionaries # tests/features/lab5.feature:6
    Given a list of dictionaries with goods # tests/features/steps/lab5_tests_BDD.py:10 0.000s
    When I extract field "title" # tests/features/steps/lab5_tests_BDD.py:26 0.000s
    Then I should get list with titles # tests/features/steps/lab5_tests_BDD.py:48 0.000s
    """
    Ковер, Диван для отдыха
    """

  Scenario: Extract multiple fields from dictionaries # tests/features/lab5.feature:14
    Given a list of dictionaries with goods # tests/features/steps/lab5_tests_BDD.py:10 0.000s
    When I extract fields "title" and "price" # tests/features/steps/lab5_tests_BDD.py:30 0.000s
    Then the result should contain 3 items # tests/features/steps/lab5_tests_BDD.py:53 0.000s

  Scenario: Remove duplicates from numbers # tests/features/lab5.feature:19
    Given a list with duplicate items # tests/features/steps/lab5_tests_BDD.py:18 0.000s
    When I get unique items # tests/features/steps/lab5_tests_BDD.py:34 0.000s
    Then I should get list with values # tests/features/steps/lab5_tests_BDD.py:42 0.000s
    """2, 3
    1, 2, 3
    """

1 feature passed, 0 failed, 0 skipped
3 scenarios passed, 0 failed, 0 skipped
9 steps passed, 0 failed, 0 skipped
Took 0min 0.001s
(base) an.akopov@MacBook-Air-Akopov lab3 %

```

Уроним тесты

```

(base) an.akopov@MacBook-Air-Akopov lab3 % behave tests/features/
USING RUNNER: behave.runner:Runner
Feature: Laboratory Functions Testing # tests/features/lab5.feature:1
  As a developer
  I want to test laboratory functions
  So that I can ensure they work correctly
  Scenario: Extract single field from dictionaries # tests/features/lab5.feature:6
    Given a list of dictionaries with goods # tests/features/steps/lab5_tests_BDD.py:10 0.000s
    When I extract field "title" # tests/features/steps/lab5_tests_BDD.py:26 0.000s
    Then I should get list with titles # tests/features/steps/lab5_tests_BDD.py:48 0.000s
    """
    Ковер, Диван для отдыха
    """
    ASSERT FAILED: Expected ['Ковер', 'Диван для отдыха'], got ['Диван для отдыха']

  Scenario: Extract multiple fields from dictionaries # tests/features/lab5.feature:14
    Given a list of dictionaries with goods # tests/features/steps/lab5_tests_BDD.py:10 0.000s
    When I extract fields "title" and "price" # tests/features/steps/lab5_tests_BDD.py:30 0.000s
    Then the result should contain 3 items # tests/features/steps/lab5_tests_BDD.py:53 0.000s

  Scenario: Remove duplicates from numbers # tests/features/lab5.feature:19
    Given a list with duplicate items # tests/features/steps/lab5_tests_BDD.py:18 0.000s
    When I get unique items # tests/features/steps/lab5_tests_BDD.py:34 0.000s
    Then I should get list with values # tests/features/steps/lab5_tests_BDD.py:42 0.000s
    """
    1, 2, 3
    """

Failing scenarios:
  tests/features/lab5.feature:6 Extract single field from dictionaries

0 features passed, 1 failed, 0 skipped
2 scenarios passed, 1 failed, 0 skipped
8 steps passed, 1 failed, 0 skipped
Took 0min 0.000s
(base) an.akopov@MacBook-Air-Akopov lab3 %

```